src\DINO\Dino_Game_basic.ino

```ino
#include <Servo.h>

// --- Constants ---
const int SERVO_START_POS = 10;           // Servo resting position
const int SERVO_ACTUATE_POS = 0;          // Servo jump position
const int SERVO_PIN = 9;                  // Servo control pin
const float DINO_ACCELERATION = 0.001;    // pixels/ms^2 (found online)
const float DINO_INITIAL_SPEED = 6.0;     // pixels/ms (found online)
const int DINO_OBSTACLE_DISTANCE = 1200;  // pixels (estimated distance to obstacle, depends on LDR placement)
const int MIN_JUMP_DELAY_MS = 120;        // Minimum jump delay to avoid negative/too-short delays
const int TAP_HOLD_MS = 200;              // Time to hold servo in jump position

const int LDR_PIN = A0;                   // LDR analog pin
const float LDR_LIGHT_THRESHOLD = 410.0;  // Threshold for light scene
const float LDR_DARK_THRESHOLD = 460.0;   // Threshold for dark scene
const int LDR_AVG_SAMPLES = 50;           // Number of samples for averaging

const int SERIAL_BAUD = 9600;             // Serial baud rate
const unsigned long FIRST_CACTUS_JUMP = 1;// First jump to start timing
const int SCORE_UPDATE_STEP = 50;         // Print score every 50 points
const int SCENE_CHANGE_DELAY_MS = 500;    // Delay for scene change
const int SCORE_TIME_OFFSET = 35;         // Offset for score timing
const int SCORE_DIVISOR = 100;            // Divisor for score calculation

// --- Global Variables ---
Servo myservo;
float ldrValue;
int jumpCount = 0;
bool shouldJump = false;                   // Flag for jump action
bool isDay = true;                         // Flag for day/night scene
unsigned long firstCactusTime;             // Time when first cactus is detected
bool cactusDetected = false;               // Flag for first cactus detection
int score = 0;                             // Initial score
int lastPrintedScore = -SCORE_UPDATE_STEP; // For reduced serial output
bool sceneChanging = false;                // Prevent tap during scene change

// --- Setup ---
void setup() {
    Serial.begin(SERIAL_BAUD);
    myservo.attach(SERVO_PIN);
    myservo.write(SERVO_START_POS);
}

// --- Main Loop ---
/*
```

```
 * First update score (to ensure accurate scene changes),
 * then handle any scene changes based on the score, read the
 * LDR value, print status if needed, and finally check
 * and handle jump actions based on the LDR reading and scene.
*/
void loop() {
    updateScore();
    handleSceneChange();
    ldrValue = getAverageLDR(LDR_AVG_SAMPLES);
    printStatus();
    checkAndHandleJump();
}

// --- Update Score ---
/*
 * This function updates the player's score based on the time elapsed
 * since the first cactus detection, plus a small offset to account for
 * the delay between detection and arrival at the dinosaur. Note that
 * the first cactus always reaches the dinosaur after 4.2 seconds, so
 * the score at that point should be 42.
 */
void updateScore() {
    if (cactusDetected) {
        unsigned long elapsedTime = millis() - firstCactusTime + SCORE_TIME_OFFSET;
        score = elapsedTime / SCORE_DIVISOR;
    }
}

// --- Handle Scene Change ---
/*
 * This function manages the transition between day and night scenes
 * based on the player's score. The first scene change occurs at
 * 700 points, switching to night. Subsequent changes occur every
 * 500 points for day to night, and 200 points for night to day. A delay
 * is introduced to prevent rapid toggling, while a sceneChanging flag
 * is used to prevent actions during the transition.
 */
void handleSceneChange() {
    static int lastSceneChangeScore = 0;
    static bool firstSceneChangeDone = false;
    if (!firstSceneChangeDone && score >= 700) {
        sceneChanging = true;
        delay(SCENE_CHANGE_DELAY_MS);
        isDay = false;
        lastSceneChangeScore = score;
        sceneChanging = false;
        firstSceneChangeDone = true;
        Serial.println("Switched to night (first change)");
    } else if (firstSceneChangeDone) {
```

```
        if (!isDay && (score - lastSceneChangeScore) >= 200) {
            sceneChanging = true;
            delay(SCENE_CHANGE_DELAY_MS);
            isDay = true;
            lastSceneChangeScore = score;
            sceneChanging = false;
            Serial.println("Switched to day");
        } else if (isDay && (score - lastSceneChangeScore) >= 500) {
            sceneChanging = true;
            delay(SCENE_CHANGE_DELAY_MS);
            isDay = false;
            lastSceneChangeScore = score;
            sceneChanging = false;
            Serial.println("Switched to night");
        }
    }
}

// --- Get Average LDR Value ---
/*
 * This function calculates the average value of the LDR readings
 * over a specified number of samples using a moving average filter.
 * Note: this was an idea I came across online, but didn't get the chance
 * to fully implement or test in this project. Thought it was a neat idea.
 */
float getAverageLDR(int samples) {
    static int readings[LDR_AVG_SAMPLES]; // buffer for moving average, size can be
adjusted
    static int index = 0;
    static int count = 0;
    int reading = analogRead(LDR_PIN);
    readings[index] = reading;
    index = (index + 1) % samples;
    if (count < samples) count++;
    long sum = 0;
    for (int i = 0; i < count; i++) {
        sum += readings[i];
    }
    return (float)sum / count;
}

// --- Print Status (Reduced Output) ---
/*
 * This function prints the current score and average LDR value
 * to the serial monitor only when the score has changed by a
 * significant step (SCORE_UPDATE_STEP) to reduce output volume.
 */
void printStatus() {
    if (score - lastPrintedScore >= SCORE_UPDATE_STEP) {
```

```
143         Serial.print("Score: ");
144         Serial.println(score);
145         Serial.print("Average light intensity: ");
146         Serial.println(ldrValue);
147         lastPrintedScore = score;
148     }
149 }
150 
151 // --- Jump Logic ---
152 /*
153  * This function checks if the dinosaur should jump based on the
154  * current scene and LDR value. If a jump is needed and no scene
155  * change is occurring, it calculates the appropriate delay based
156  * on the dinosaur's speed and distance to the obstacle, then
157  * performs the jump action.
158  */
159 void checkAndHandleJump() {
160     shouldJump = isDay ? (ldrValue >= LDR_DARK_THRESHOLD) : (ldrValue <=
    LDR_LIGHT_THRESHOLD);
161     if (shouldJump && !sceneChanging) {
162         Serial.println("Jumping!");
163         int jumpDelay = calculateJumpDelay();
164         delay(jumpDelay);
165         tap();
166         jumpCount++;
167         recordFirstCactus();
168     }
169 }
170 
171 /*
172  * This function calculates the delay before the dinosaur jumps
173  * based on its current speed and the distance to the obstacle.
174  */
175 int calculateJumpDelay() {
176     unsigned long elapsedTime = millis() - firstCactusTime;
177     if (elapsedTime < 0) elapsedTime = 0; // Prevent negative elapsed time
178     float currentSpeed = DINO_INITIAL_SPEED + DINO_ACCELERATION * elapsedTime;
179     int jumpDelay = (int)(DINO_OBSTACLE_DISTANCE / currentSpeed);
180     if (jumpDelay < MIN_JUMP_DELAY_MS) jumpDelay = MIN_JUMP_DELAY_MS;
181     return jumpDelay;
182 }
183 
184 /*
185  * This function performs the tap action to make the dinosaur jump.
186  */
187 void tap() {
188     myservo.write(SERVO_ACTUATE_POS); // Move servo to jump position
189     delay(TAP_HOLD_MS);               // Hold position briefly
190     myservo.write(SERVO_START_POS);   // Return to start position
```

```
}

/*
 * This function records the time when the first cactus is detected,
 * which is indicated by the first jump action. This time is used
 * to calculate the score based on elapsed time since this event.
 */
void recordFirstCactus() {
    if (!cactusDetected && jumpCount >= FIRST_CACTUS_JUMP) {
        firstCactusTime = millis() - SCORE_TIME_OFFSET;
        cactusDetected = true;
        Serial.print("First cactus detected, beginning timing: ");
        Serial.println(firstCactusTime);
    }
}


```